

中山大學
SUN YAT-SEN UNIVERSITY

Lecture 15
Binary Trees

Data Structures and Algorithms

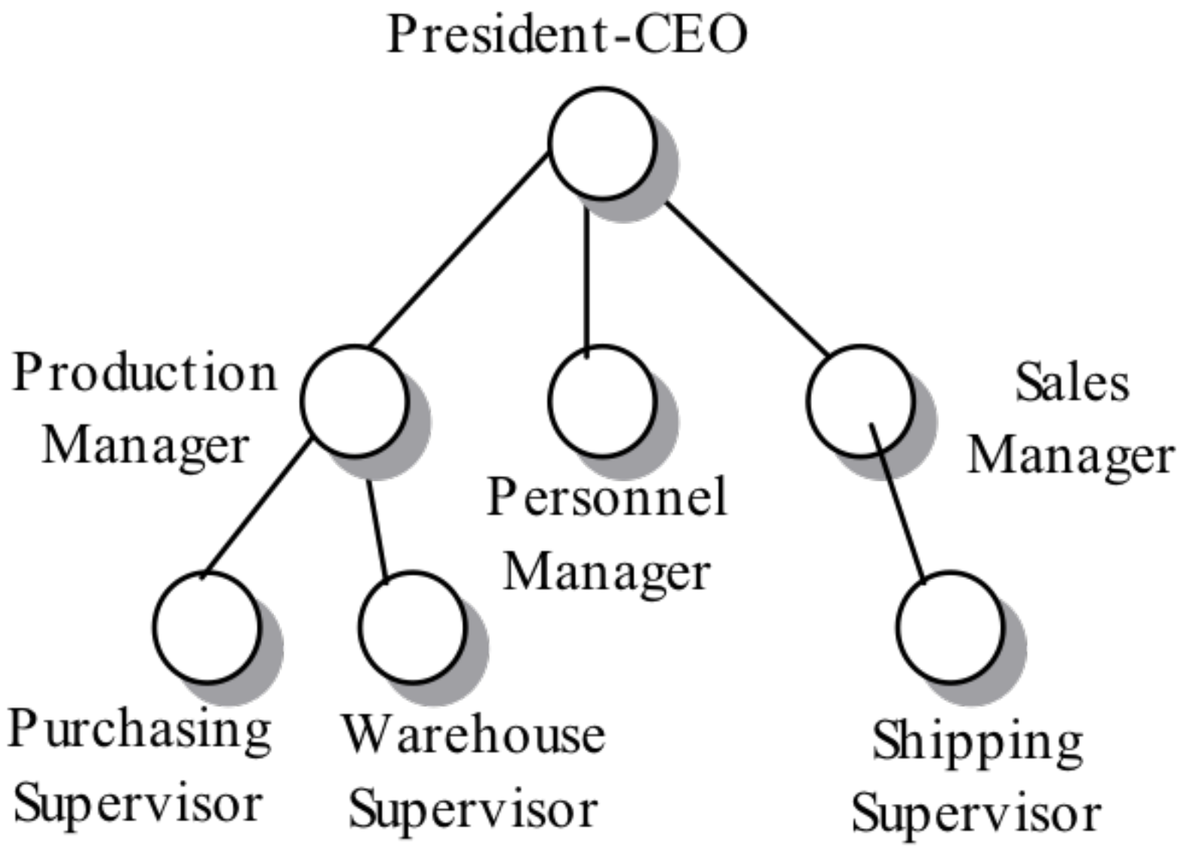
Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Outline

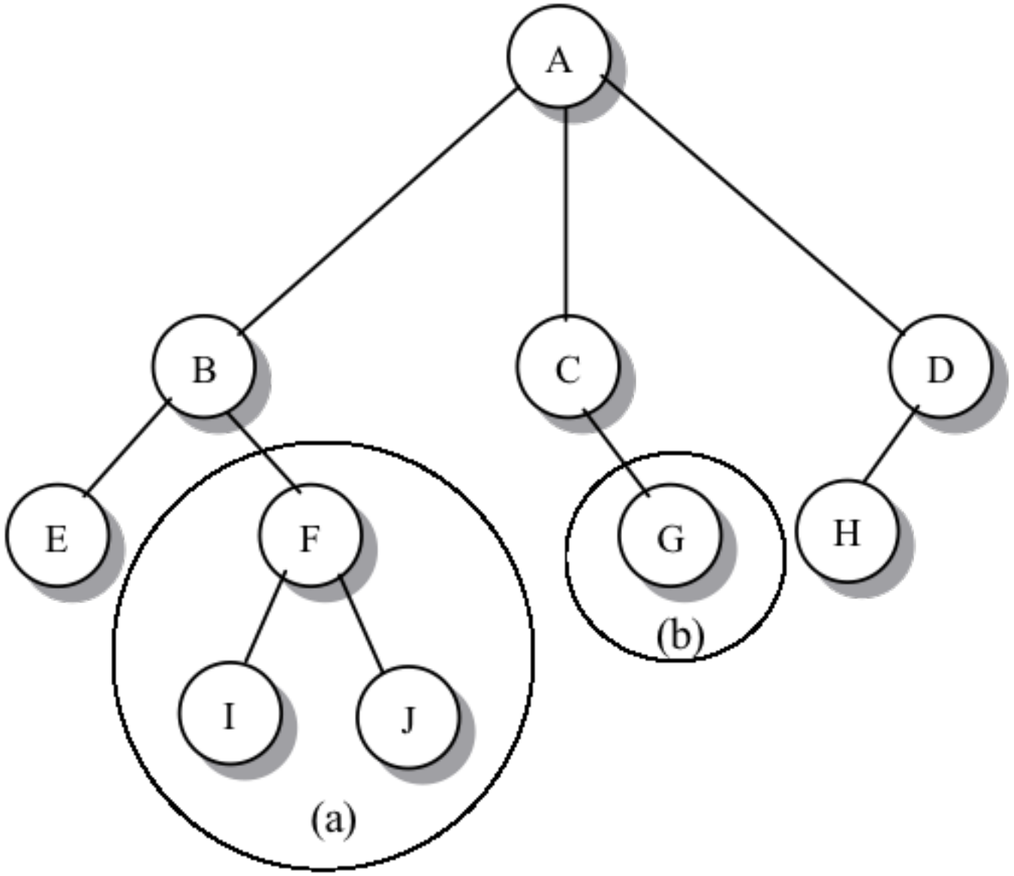
- Tree Structure
- Binary Trees
  - Definition
  - Implementation of Binary Trees
  - Traversal of Binary Trees

Tree Structure



HIERARCHICAL TREE STRUCTURE

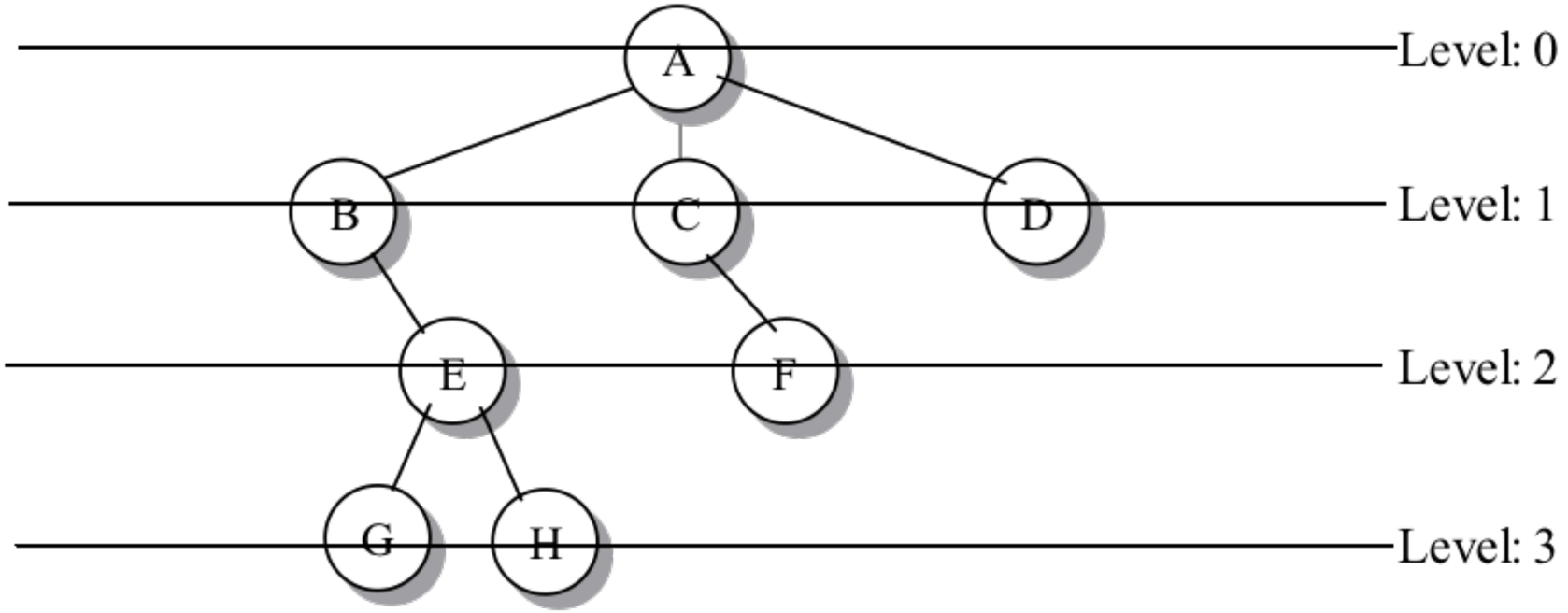
Terminologies



A GENERAL TREE

root, child, parent,
leaves, siblings, subtree,
grandparent, grandchild,
ancestor, descendant

Terminologies

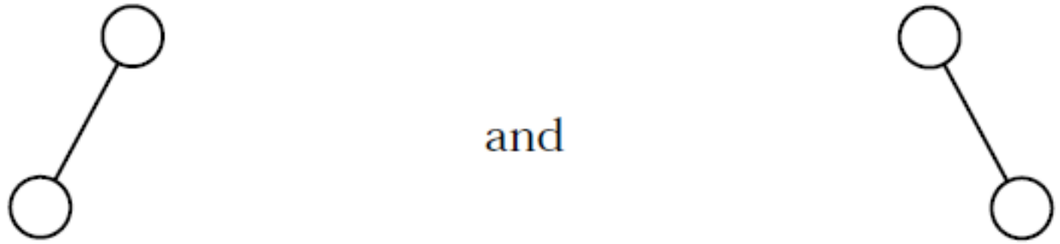


- A ***path*** from node n_1 to n_k is defined as a sequence of nodes n_1, n_2, ..., n_k such that n_i is the parent of n_{i+1} for $1 \le i < k$. The ***length*** of this path is the number of edges on the path, namely k-1.
- For any node n_i, the ***depth*** of n_i is the length of the unique path from the root to n_i. The root is at depth 0. The ***height*** of n_i is the length of the longest path from n_i to a leaf. All leaves are of height 0. The height of a tree is equal to the height of the root.

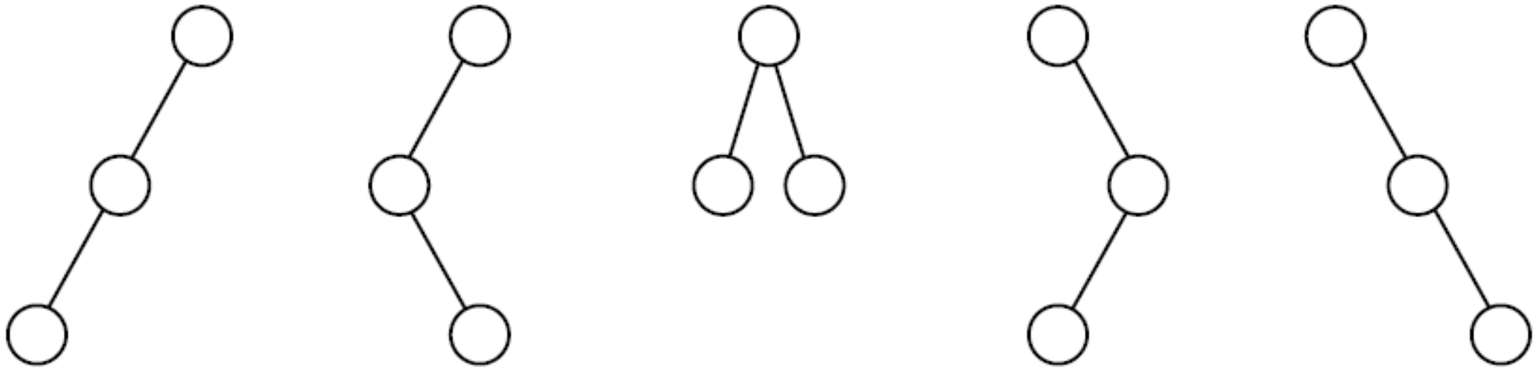
Binary Trees

DEFINITION A ***binary tree*** is either empty, or it consists of a node called the ***root*** together with two binary trees called the ***left subtree*** and the ***right subtree*** of the root.

- There is one empty binary tree, one binary tree with one node, and two with two nodes:

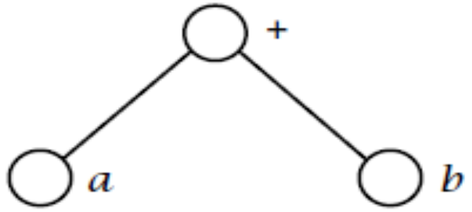


- The binary trees with three nodes are:

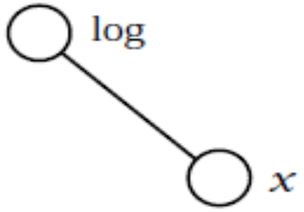


Expression Trees

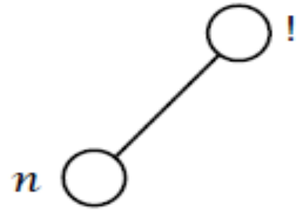
- ***An expression tree*** is a binary tree such that every node is labeled by a binary operator or unary operator, its left subtree denotes its left operand and right subtree its right operand.



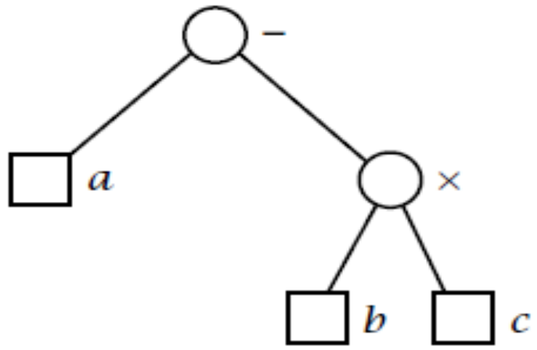
$a + b$



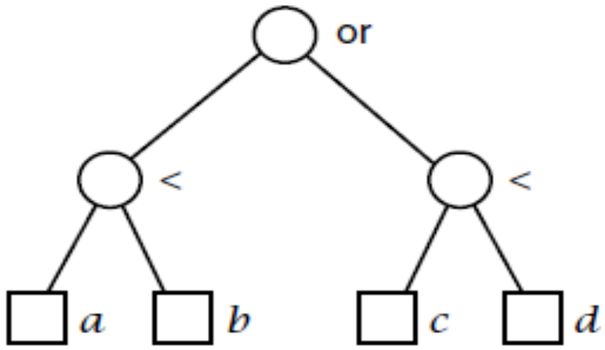
$\log x$



$n!$



$a - (b \times c)$

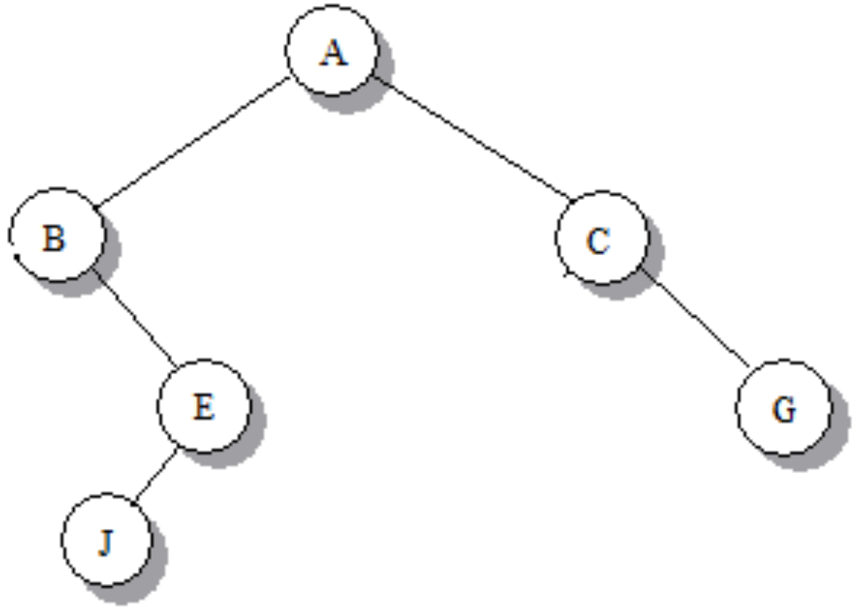


$(a < b)$ or $(c < d)$

Binary Trees: Sequential Storage

- The root of the tree is in position 0 of the list, then the left and right children of the node in position *k* are in positions 2*k*+1 and 2*k*+2 of the list, respectively.

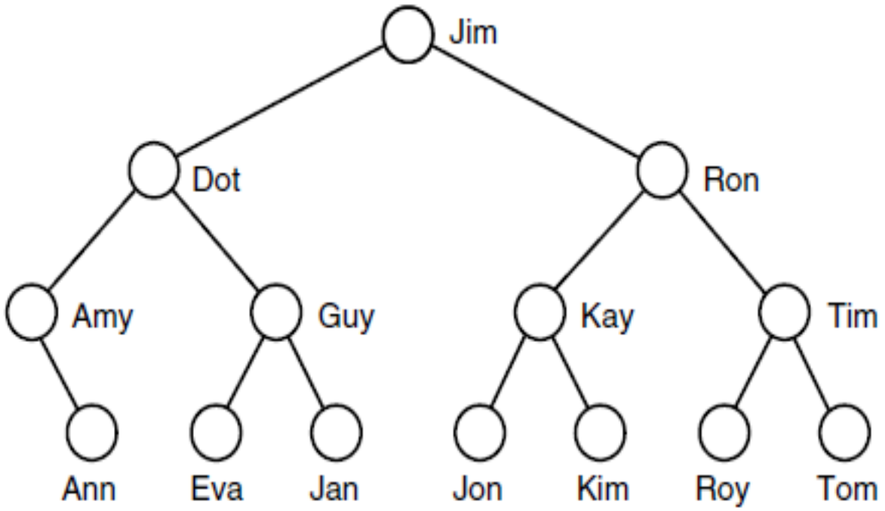
0	1	2	3	4	5	6	7	8	9
A	B	C	^	E	^	G	^	^	J

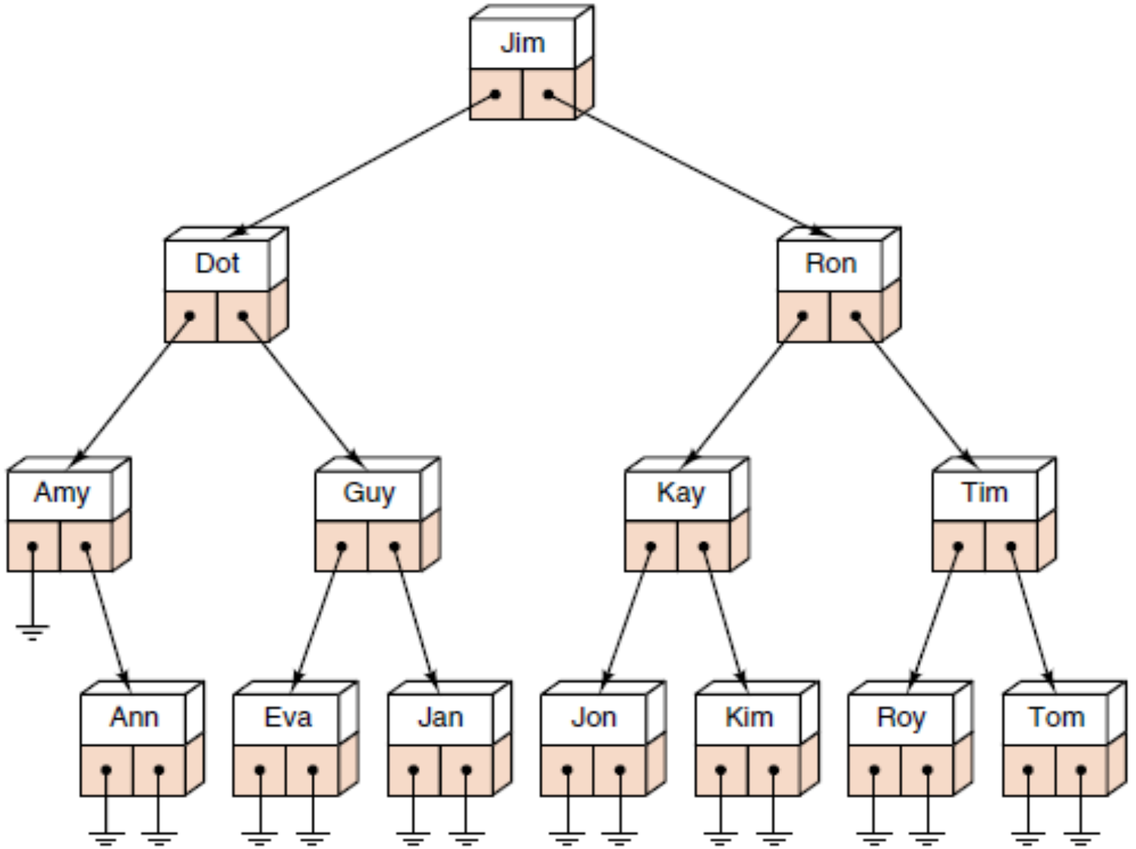


- The tree of height *k* needs 2^k spaces. Not effective!

Linked Implementation of Binary Trees

- A natural implementation is in linked storage. A node has two pointers that denote its left and right subtrees.



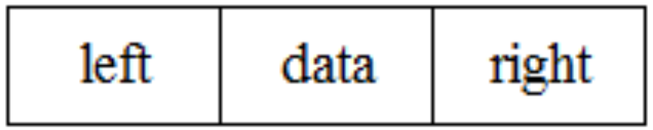


Linked Binary Tree Specifications

Binary tree class:

```
template <class Entry>
class Binary_tree {
public:
//    Add methods here.
protected:
   //    Add auxiliary function prototypes here.
   Binary_node<Entry> *root;
};
```

Node structure:



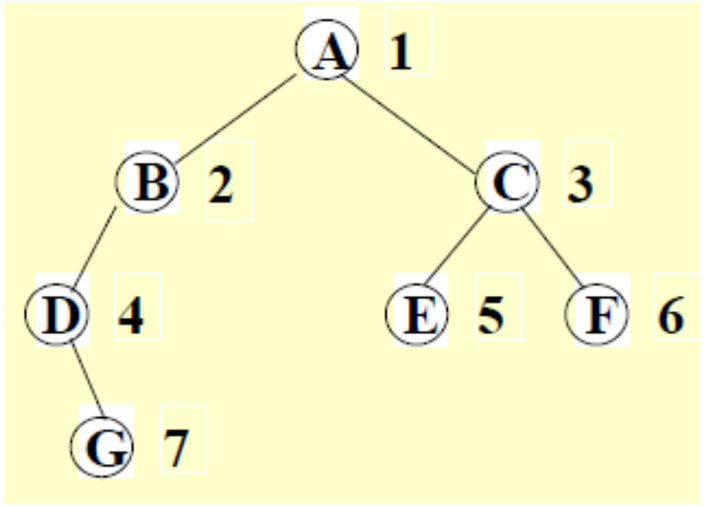
Binary node class:

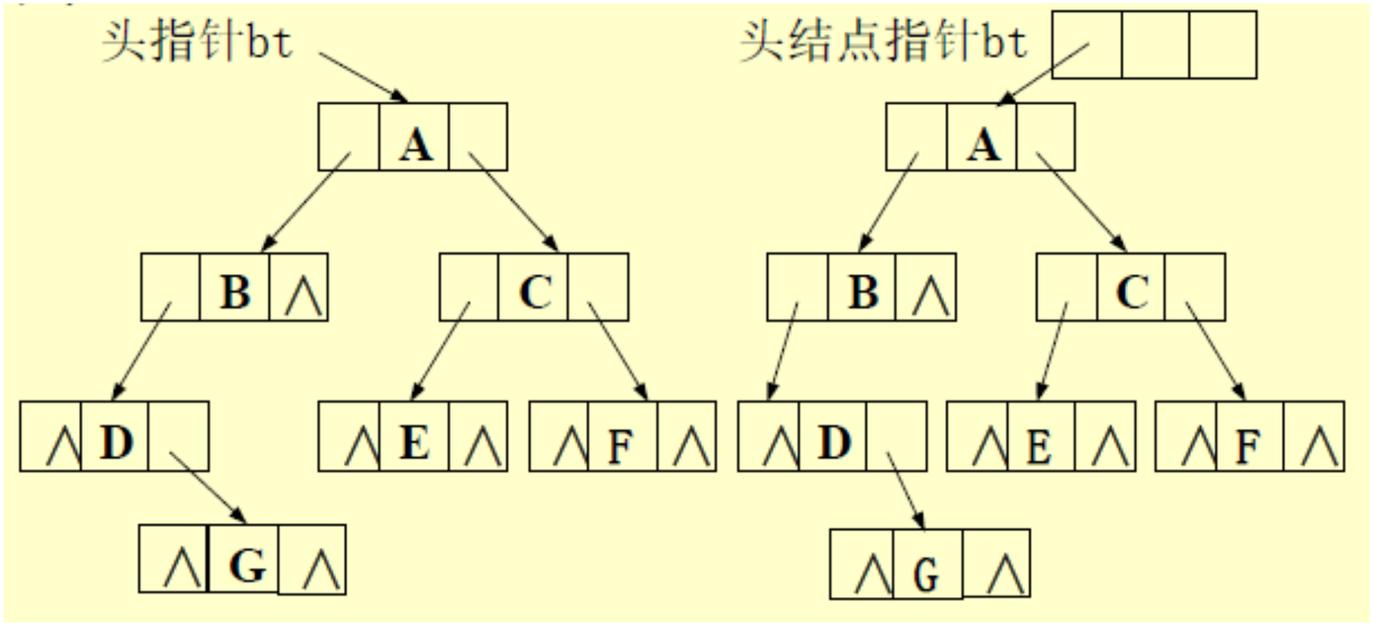
```
template <class Entry>
struct Binary_node {

//    data members:
   Entry data;
   Binary_node<Entry> *left;
   Binary_node<Entry> *right;
//    constructors:
   Binary_node( );
   Binary_node(const Entry &x);


};
```

Binary Tree with/without Head Node





Traversal of Binary Trees

- At a given node there are three tasks to do in some order: Visit the node itself (*V)*; traverse its left subtree *(L);* traverse its right subtree (*R).* There are six ways to arrange these tasks:

  (V L R) (L V R) (L R V) (V R L) (R V L) (R L V)

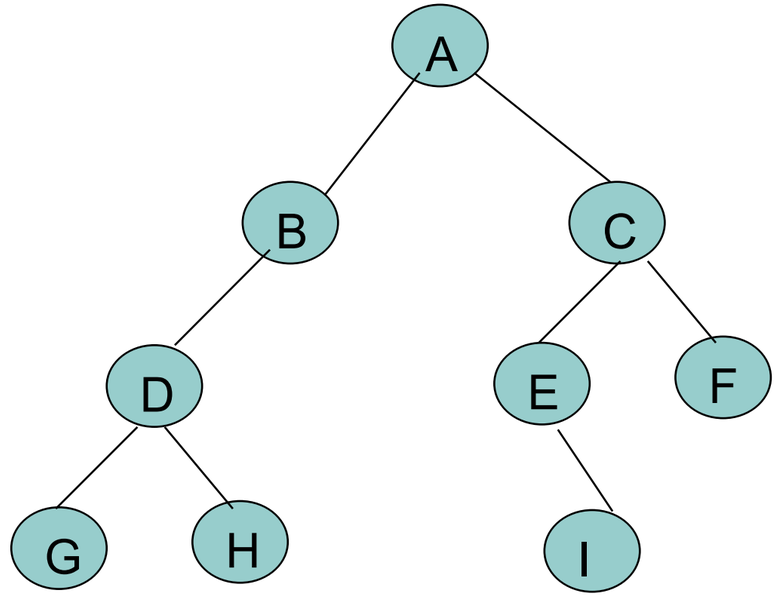
- By standard convention, these are reduced to three by considering only the ways in which the left subtree is traversed before the right.

$V\,L\,R$	$L\,V\,R$	$L\,R\,V$
preorder	*inorder*	*postorder*

- These three names are chosen according to the step at which the given node is visited.

Tree Traversal (in-order)

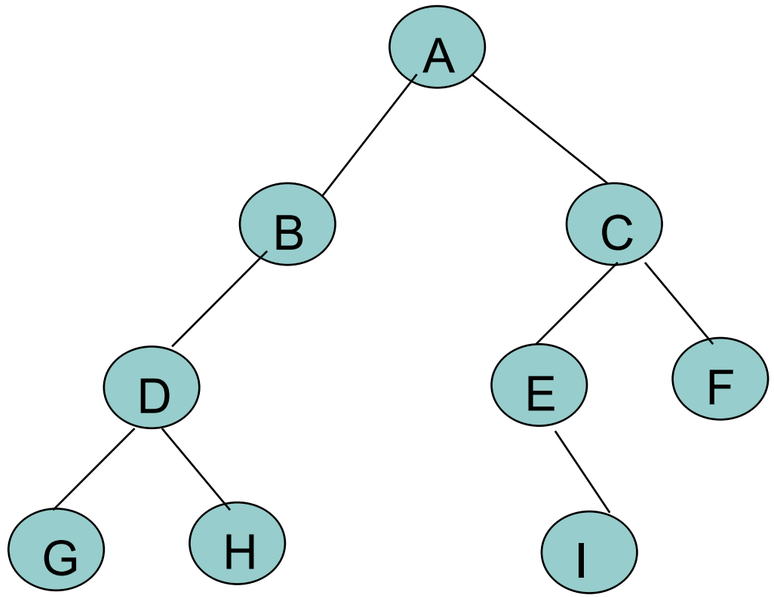
```
template <class Entry>
void Binary_tree<Entry> :: recursive_inorder(Binary_node<Entry> *sub_root,
                                             void (*visit)(Entry &))
/* Pre:   sub_root is either NULL or points to a subtree of the Binary_tree.
   Post:  The subtree has been been traversed in inorder sequence.
   Uses:  The function recursive_inorder recursively */
{
   if (sub_root != NULL) {
      recursive_inorder(sub_root->left, visit);
      (*visit)(sub_root->data);
      recursive_inorder(sub_root->right, visit);
   }
}
```



in-order: G, D, H, B, A, E, I, C, F

Tree Traversal (pre-order)

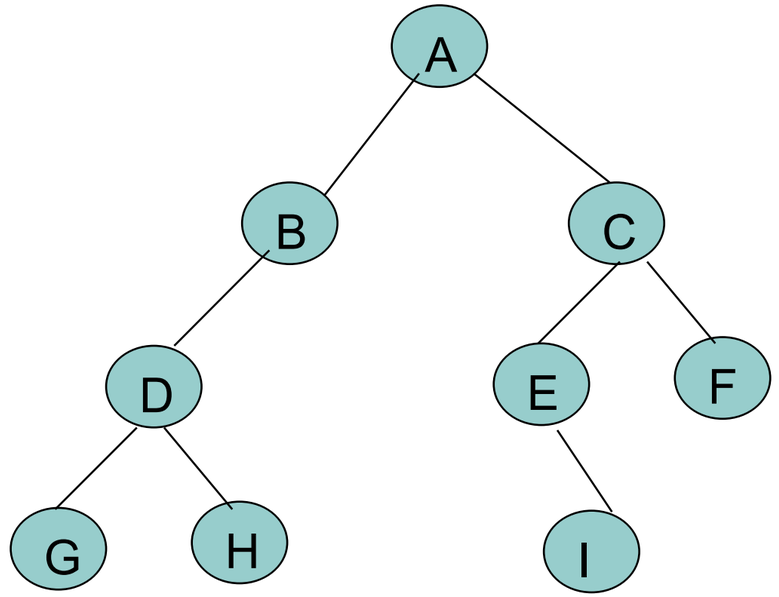
```
template <class Entry>
void Binary_tree<Entry> :: recursive_preorder(Binary_node<Entry> *sub_root,
                                              void (*visit)(Entry &))
/* Pre:   sub_root is either NULL or points to a subtree of the Binary_tree.
   Post:  The subtree has been been traversed in preorder sequence.
   Uses:  The function recursive_preorder recursively */
{
   if (sub_root != NULL) {
      (*visit)(sub_root->data);
      recursive_preorder(sub_root->left, visit);
      recursive_preorder(sub_root->right, visit);
   }
}
```



pre-order: A, B, D, G, H, C, E, I, F

Tree Traversal (post-order)

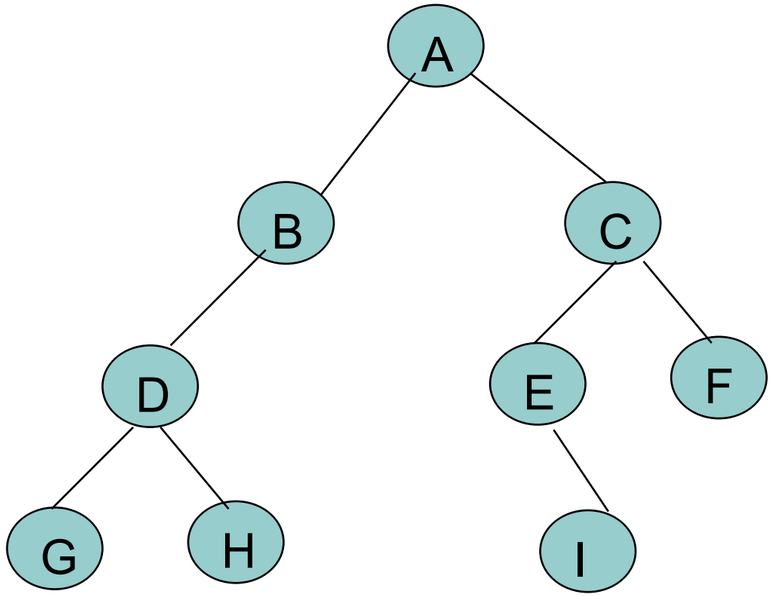
```
template <class Entry>
void Binary_tree<Entry> :: recursive_postorder(Binary_node<Entry> *sub_root,
                                               void (*visit)(Entry &))
/* Pre:   sub_root is either NULL or points to a subtree of the Binary_tree.
   Post:  The subtree has been been traversed in postorder sequence.
   Uses:  The function recursive_postorder recursively */
{
   if (sub_root != NULL) {
      recursive_postorder(sub_root->left, visit);
      recursive_postorder(sub_root->right, visit);
      (*visit)(sub_root->data);
   }
}
```



post-order: G, H, D, B, I, E, F, C, A

Tree Traversal (level-by-level)

- 层次遍历：若树为空，则空操作返回，否则从树的第0层，也就是根节点开始访问，从上而下逐层遍历，在同一层中，按从左到右手的顺序对结点逐个访问。



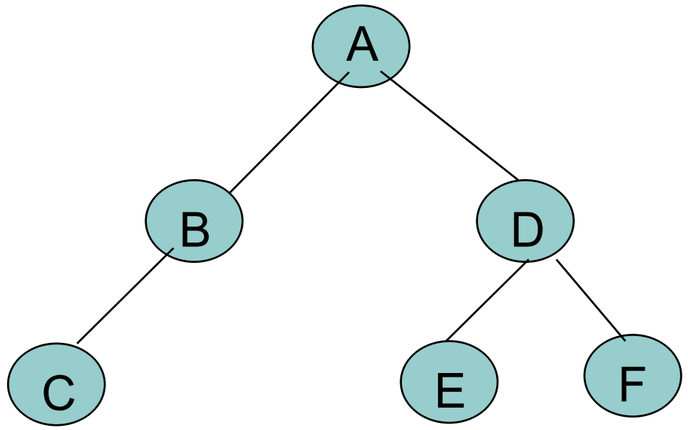
Level traversal: A, B, C, D, E, F, G, H, I

Tree Traversal (level-by-level)

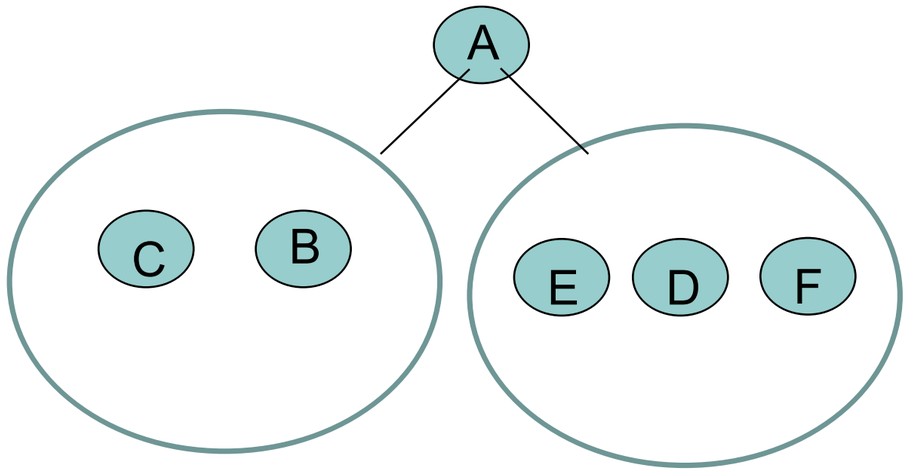
```
struct BinaryNode {
    int data;
    BinaryNode *left, *right;
};
void traversal(BinaryNode *p)
{
    queue<BinaryNode*> q;
    BinaryNode *cur;
    if (p != NULL) q.push(p);
    while (!q.empty()) {
        cur = q.front();
        q.pop();
        printf("%d ", cur->data);
        if (cur->left != NULL) q.push(cur->left);
        if (cur->right != NULL) q.push(cur->right);
    }
}
```

Construct Binary Tree from Different Traversal Orders

- Given inorder and postorder traversal of a tree, construct the binary tree.

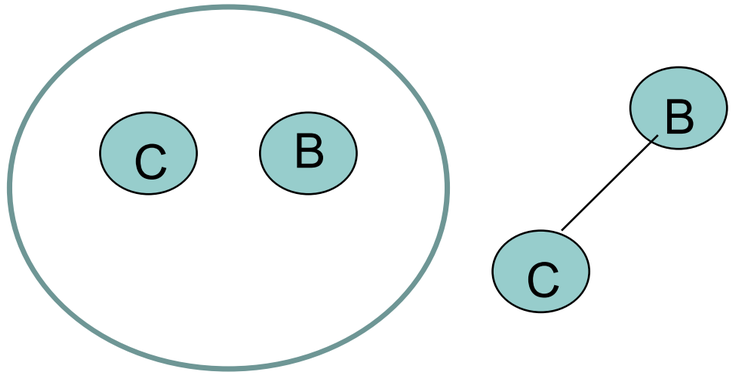


- Preorder: ABCDEF → (A)(BC)(DEF)
- Inorder: CBAEDF → (CB)(A)(EDF)

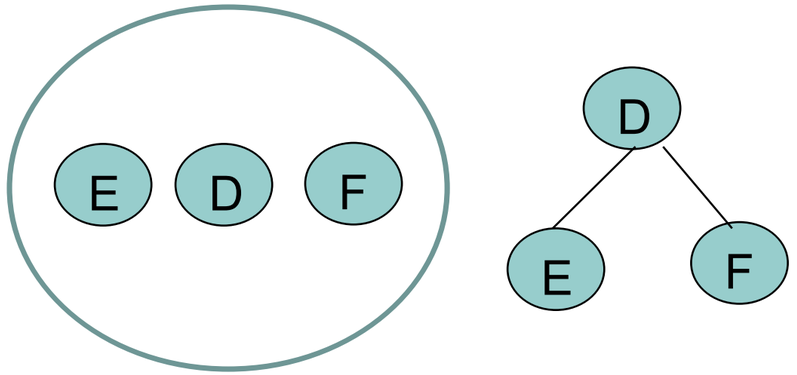


Construct Binary Tree from Different Traversal Orders

- Left Subtree
- Preorder: BC → (B)(C)
- Inorder: CB → (C)(B)



- Right Subtree
- Preorder: DEF → (D)(E)(F)
- Inorder: EDF → (E)(D)(F)



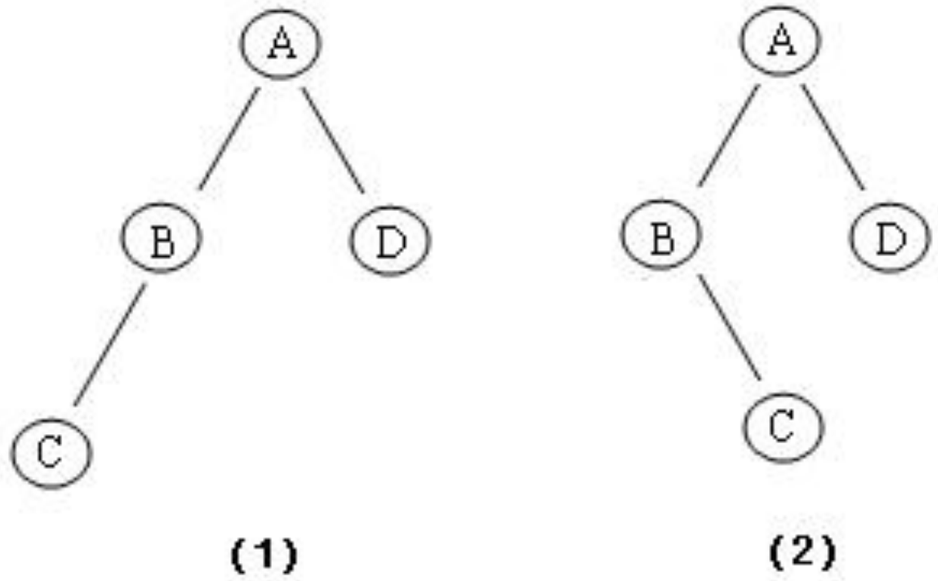
- Exercise:

  Preorder: ABCDEFGHI

  Inorder: BCAEDGHFI

Construct Binary Tree from Different Traversal Orders

- Given preorder and postorder traversal of a tree, construct the binary tree.
- May Not Be Unique!
- Example:
- Preorder: ABCD
- Postorder: CBDA



Thank you!

